

**Program and Branch Name:**

BTech (CSE)

PBL Progress Report – II

Field	Details
Course Name	Artificial Intelligence
Course Code	CCSAI0301
Faculty Name	Dr. Mohd Nazim
Assignment Type	Group Assignment
Student(s) Name	Maneesh Ray, Chandan Kumar, Devanshu, Sanjit Kumar, Amresh Singh
ERP ID (I Year) / Roll No. (II Year onwards)	0261DCS006, 0261DCS003, 0261DCS016, 0261DCS013, 0261DCS005
Team/Group No. (<i>Group/Capstone only</i>)	Group 98
Problem Statement / Project Title	AI-Based Phishing Email Detection & Decision Support System
SDG(s)	SDG 9 – Industry, Innovation & Infrastructure
Reporting Month	Month 2
Date of Submission	08/10/2026

Student Declaration

We certify that this assignment is our own work, with all sources appropriately acknowledged. We understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature(s): Maneesh Ray**Date:** 08/10/2026

PBL Progress Report – II

Review 2: Development & Application

Review Coverage:

- **Individual / Group Assignment:** Unit/Module 3 and Unit/Module 4 (plus Unit/Module 1 and 2 concepts from Month 1, now implemented)

Field	Details		
1. Previous Review Feedback	Report I left the faculty evaluation to Moodle and set this plan for the next review, which is treated as the feedback to act on: begin data pre-processing and feature engineering, implement a baseline heuristic/search-based detector, start Module 3 (Knowledge Representation) and Module 4 (Statistical Reasoning), and prepare interim results.		
2. Action Taken on Previous Feedback	• Pre-processing and feature engineering done: 7 features are extracted from raw email text. • Baseline heuristic detector and search-based parts implemented: BFS, DFS, iterative deepening, bidirectional, Greedy best-first, A*, AO*, constraint satisfaction (Module 1) and minimax with alpha-beta pruning (Module 2). • Module 3 implemented: propositional logic with resolution, first-order rules, semantic net, frames and an expert system. Module 4 implemented: Naive Bayes. • Interim results prepared on a sample dataset; see Results.		
3. Research / Data Collection Progress	• Studied the Module 1–3 topics in the syllabus (search, AO*, CSP, minimax, logic, semantic nets, frames, expert systems) and Module 4 (Bayes’ theorem, Naive Bayes). • Dataset: the Kaggle phishing email/URL dataset shortlisted in Month 1 is kept for the final stage. To build and test the pipeline, 200 sample emails (100 phishing, 100 legitimate; 140 train, 60 test) and a small URL redirect graph were generated by our script. It is sample data, not real mail. • Research papers from Month 1 (still the main references): (1) Salahdine, El Mrabet & Kaabouch (2022), “Phishing Attacks Detection – A Machine Learning-Based Approach” – selects 10 features from 4,000+ real phishing emails and compares ML classifiers; it guides the feature scoring. (2) Kamran, Sengupta & Tavakkoli (2021), “Semi-supervised Conditional GAN for Phishing URL Detection: A Game-Theoretic Perspective” – models attacker and detector as two players; it guides the adversarial search.		
4. Unit/Module Concepts Applied	Syllabus concepts applied in the project:		
	Module	Topic / Concept	Application in Project
	Month 1 plan	Pre-processing, heuristic scoring	7 binary features from raw email; weighted score gives the baseline verdict
	Module 1	BFS, DFS, iterative deepening, bidirectional, Greedy best-first, A*	Trace the redirect chain from an email link to a known phishing page
	Module 1	AO* search	Cheapest investigation plan over the rule base AND-OR graph
	Module 1	Constraint satisfaction	Assign flagged emails to analysts under constraints
	Module 2	Minimax, alpha-beta pruning	Attacker hides indicators, detector raises weights
	Module 3	Propositional logic, resolution	Prove the phishing verdict from facts and rules
	Module 3	First-order logic	Rules with variables to find look-alike sender domains
	Module 3	Semantic net, frames	Indicator taxonomy with inheritance; email record with defaults
	Module 3	Expert system, forward/backward chaining	Knowledge base, inference engine and explanation facility
Module 4	Bayes' theorem, Naive Bayes	Probability of phishing; accuracy, recall, 5-fold cross-validation	
5. Methodology / Approach Followed	1. Extract features and score them (baseline). 2. Search redirect chains, plan checks with AO*, allocate analysts with CSP (Module 1). 3. Play the attacker-detector game (Module 2). 4. Encode knowledge as logic, semantic net and frames inside an expert system that explains its verdict (Module 3). 5. Train Naive Bayes and combine it with the rules (Module 4). 6. Compare all detectors on the 60 test emails.		

6. Work Completed During Month 2	Wrote phishing_ai.py (Python 3, about 590 lines) covering all the modules above, generated sample data, ran everything and cross-checked results (resolution against forward chaining, alpha-beta against minimax).																				
7. Analysis / Development / Implementation	Implementation: Python 3, standard library only; features come from regular expressions on the sender, reply-to, URLs and attachments (weights 1 to 3). Modules 1–2: 9-URL redirect graph with risk heuristics; AO* over the rule AND-OR graph; CSP with backtracking, minimum-remaining-values and forward checking; a 4-ply attacker-detector game. Module 3: 13 Horn rules (also as clauses for resolution); first-order rules with unification; frames with an if-needed procedure; expert system with a proof tree. Module 4: Naive Bayes in log space with Laplace smoothing.																				
8. Results / Interim Outcomes	Results on the sample data (60 test emails: 28 phishing, 32 legitimate): <table border="1"> <thead> <tr> <th>Analysis</th><th>Result</th></tr> </thead> <tbody> <tr> <td>Detectors (accuracy / recall / false +)</td><td>Heuristic 0.883 / 1.000 / 7; rule base 0.867 / 1.000 / 8; Naive Bayes 0.883 / 1.000 / 7 (5-fold CV mean 0.890); combined 0.867 / 1.000 / 8</td></tr> <tr> <td>Search: redirect chain</td><td>BFS, iterative deepening, bidirectional: 2 hops, cost 5. DFS: 4 hops, cost 9. Greedy and A*: 3 hops, cost 4, the cheapest</td></tr> <tr> <td>AO* plan</td><td>Cheapest investigation checks urgency words and IP in URL, cost 5 (4 expansions)</td></tr> <tr> <td>CSP triage</td><td>8 flagged emails assigned to 3 analysts with all constraints met (15 assignments tried)</td></tr> <tr> <td>Minimax / alpha-beta</td><td>Same value (–3: the attacker cannot evade); 705 nodes against 184 with pruning; best first move: hide the reply-to mismatch</td></tr> <tr> <td>Propositional resolution</td><td>Agrees with forward chaining on 60 of 60 test emails (29 proved phishing)</td></tr> <tr> <td>First-order rules</td><td>5 look-alike sender domains found, all phishing (for example an “amazon” spoof)</td></tr> <tr> <td>Semantic net, frames, expert system</td><td>SpearPhishing inherits 3 indicators from PhishingEmail; frame computes risk score 8 on demand; the expert system returns Phishing with the proof (R6 from R1 and R3)</td></tr> <tr> <td>Naive Bayes evidence</td><td>Strongest signs of phishing: IP-address URL, reply-to mismatch, executable attachment</td></tr> </tbody> </table> The data is generated, so these figures show that the pipeline works, not real-world accuracy.	Analysis	Result	Detectors (accuracy / recall / false +)	Heuristic 0.883 / 1.000 / 7; rule base 0.867 / 1.000 / 8; Naive Bayes 0.883 / 1.000 / 7 (5-fold CV mean 0.890); combined 0.867 / 1.000 / 8	Search: redirect chain	BFS, iterative deepening, bidirectional: 2 hops, cost 5. DFS: 4 hops, cost 9. Greedy and A*: 3 hops, cost 4, the cheapest	AO* plan	Cheapest investigation checks urgency words and IP in URL, cost 5 (4 expansions)	CSP triage	8 flagged emails assigned to 3 analysts with all constraints met (15 assignments tried)	Minimax / alpha-beta	Same value (–3: the attacker cannot evade); 705 nodes against 184 with pruning; best first move: hide the reply-to mismatch	Propositional resolution	Agrees with forward chaining on 60 of 60 test emails (29 proved phishing)	First-order rules	5 look-alike sender domains found, all phishing (for example an “amazon” spoof)	Semantic net, frames, expert system	SpearPhishing inherits 3 indicators from PhishingEmail; frame computes risk score 8 on demand; the expert system returns Phishing with the proof (R6 from R1 and R3)	Naive Bayes evidence	Strongest signs of phishing: IP-address URL, reply-to mismatch, executable attachment
Analysis	Result																				
Detectors (accuracy / recall / false +)	Heuristic 0.883 / 1.000 / 7; rule base 0.867 / 1.000 / 8; Naive Bayes 0.883 / 1.000 / 7 (5-fold CV mean 0.890); combined 0.867 / 1.000 / 8																				
Search: redirect chain	BFS, iterative deepening, bidirectional: 2 hops, cost 5. DFS: 4 hops, cost 9. Greedy and A*: 3 hops, cost 4, the cheapest																				
AO* plan	Cheapest investigation checks urgency words and IP in URL, cost 5 (4 expansions)																				
CSP triage	8 flagged emails assigned to 3 analysts with all constraints met (15 assignments tried)																				
Minimax / alpha-beta	Same value (–3: the attacker cannot evade); 705 nodes against 184 with pruning; best first move: hide the reply-to mismatch																				
Propositional resolution	Agrees with forward chaining on 60 of 60 test emails (29 proved phishing)																				
First-order rules	5 look-alike sender domains found, all phishing (for example an “amazon” spoof)																				
Semantic net, frames, expert system	SpearPhishing inherits 3 indicators from PhishingEmail; frame computes risk score 8 on demand; the expert system returns Phishing with the proof (R6 from R1 and R3)																				
Naive Bayes evidence	Strongest signs of phishing: IP-address URL, reply-to mismatch, executable attachment																				
9. Evidence / Supporting Material	- Source code phishing_ai.py and saved console output phishing_output.txt. - Project repository: LINK																				
10. Challenges & Corrective Actions	- Public datasets have inconsistent labels, so generated data with known features was used to test the pipeline first. - Single indicators cause false alarms (7–8 of 32 legitimate emails flagged), so rules and probability are used together; thresholds need tuning on real data. - Classic puzzle problems in the syllabus (water jug, queens, travelling salesperson, missionaries and cannibals, tiles) do not map to phishing detection and were not included. - Limitation: sample data only; results must be repeated on the Kaggle dataset.																				
11. Individual / Team Contribution	Maneesh Ray (Team Lead & AI Solution Architect): design and integration of the pipeline. Chandan Kumar (Search Algorithm & Optimisation Analyst): search algorithms, AO* and heuristic weights. Devanshu (Heuristic & Decision Intelligence Specialist): decision logic and adversarial search. Sanjit Kumar (Knowledge Representation & Expert System Designer): logic, rule base, semantic net, frames, expert system. Amresh Singh (Statistical AI Analyst & Documentation Lead): Naive Bayes, evaluation, Month 2 report.																				
12. Overall Progress (%)	60% – Month 1 (20%) covered topic, team, dataset shortlisting and problem framing; Month 2 added the working feature pipeline and the Module 1–4 detectors with interim results.																				
13. Plan for Final Review	- Run the pipeline on the Kaggle dataset; tune weights, rules and thresholds to cut false positives. - Build the explainable decision output (verdict, reasons, confidence) and finish the final report and demo.																				

Faculty Evaluation:

- To be completed by Faculty in Moodle